Vapor Stone
RAIL SYSTEMS
A Wabtec company

10 655 Henri-Bourassa Blvd., West
Saint-Laurent (Québec) Canada
H4S 1A1

Tel: 514.335.4200
Fax: 514.335.4231

www.wabtec.com

Vapor Stone Rail Systems

Generic Software Documentation Project

Project # 7260

Door System

Software Design Description (SDD)
Neuron Serial Peripheral Interface (SPI) Module

Contract Number		CDRL Number	
		Customer Ref. Number	

NOTICE
Plans, specifications and engineering data submitted herewith or which may be submitted, remain the property of **Vapor Stone Rail Systems** and shall not be reproduced or altered or used in whole or in part except by express consent of **VSRS**.

AVIS
Les plans, spécifications et données techniques soumis ci-joints, ou qui peuvent être soumis, restent la propriété de **Vapor Stone Rail Systems** et ne doivent pas être reproduits, altérés ou utilisés, totalement ou partiellement, sans le consentement express de **VSRS**.

	Name / Title	Signature / Date
Prepared :	James Kim Software Designer	James Kim 2011-10-06
Verified :	Martin Ouellet, Eng. Software Department Supervisor	2011-10-06
Approved :	David Beaudoin, Eng. Soft. and Elect. Engineering Manager	2011-10-07

Document No.	**201111659**	Revision	**1**
		Date (yyyy-mm-dd)	**2011-10-06**

Revision List

Rev.	Date	By	ECD	Description of changes
1	2011-10-06	James Kim	---	Official release
0.1	2011-10-05	James Kim	---	Implementing comments from internal inspection I-1853.
0	2011-10-03	James Kim	---	Original issue

Preface

This Software Design Description (SDD) complies with the requirements of the IEEE 1016-1998 and IEEE 1558-2004.

Table of Contents

Revision List 2
Preface 2
Table of Contents 3
List of Figures 4
List of Tables 4

1 INTRODUCTION 5

1.1 Purpose 5
1.2 Scope 5
1.3 Definitions and Acronyms 5
1.3.1 Definitions 5
1.3.2 Acronyms 5
1.4 Document Organization 6

2 REFERENCES 7

3 SOFTWARE DESIGN 8

3.1 Decomposition Description 8
3.1.1 Module Decomposition 8
3.1.2 Concurrent Process Decomposition 8
3.1.3 Data Decomposition 8
3.2 Dependency Description 9
3.2.1 Inter-Module Dependencies 9
3.2.2 Inter-Process Dependencies 9
3.2.3 Data Dependencies 9
3.3 Interface Description 10
3.3.1 Module Interfaces 10
3.4 Detailed Design 11
3.4.1 Neuron SPI Module Detailed Design 11
3.4.2 Data Entities Detailed Design 16

List of Figures

Figure 3-1: Inter-Module Dependencies Diagram 9
Figure 3-2: Inter-Process Dependencies Diagram 9
Figure 3-3: SPI Port Block Diagram 12
Figure 3-4: Neuron SPI Interface Process 12
Figure 3-5: spi_IsrRx Flowchart 13
Figure 3-6: spi_IsrTx Flowchart 13
Figure 3-7: spi_TaskDecodePacket Flowchart 15
Figure 3-8: spi_TaskEncodePacket Flowchart 16

List of Tables

Table 3-1: SPI_Frame Message from Neuron to DCM 8
Table 3-2: SPI_Frame Message from DCM to Neuron 8
Table 3-3: PEC channels Initialization for SPI Port 11

1 INTRODUCTION

This document is the Software Design Description of the Serial Peripheral Interface (SPI) module developed by Vapor Stone Rail Systems.

The Neuron SPI Module is a part of the Door Control Module (DCM) Application that interfaces with the Neuron-based application.

1.1 Purpose

The purpose of this Software Design Description (SDD) is to describe the software components and required implementation of the Neuron SPI Module.
The intended audience is the Vapor Stone Rail Systems software development team and maintenance team members of the transit authority and the car builder.

1.2 Scope

The Neuron SPI Module is a software module used for interfacing with the Neuron processor.
This module interfaces with other modules.

1.3 Definitions and Acronyms

1.3.1 Definitions

Door Control Module: A specific processor based electronic module running a Door Control Module Application. It is used for controlling and monitoring the door.

1.3.2 Acronyms

Acronym	Meaning
ASCII	American Standard Code for Information Interchange
CDRL	Contract Data Requirements List
CL	Car Line
CPLD	Complex Programmable Logic Device
CPU	Central Processing Unit
DCM	Door Control Module
DCU	Door Control Unit
DSR	Door Summary Relay
ECD	Engineering Change Document
FLOG	Fault Logger
IEEE	Institute of Electrical and Electronics Engineers
IO	Input and Output
LH	Left Hand
LSS	Lock Sensing Switch
MHz	MegaHertz
MONIT	Monitoring
ms	Millisecond
NVRAM	Non-Volatile Random Access Memory
OC	Over Current
OS	Operating System
PEC	Peripheral Event Controller
PID	Proportional Integral Derivative
PSS	Panel Sensing Switch
PT	Portable Test
PWM	Pulse Width Modulation
PWR	Power
RAM	Random Access Memory
RH	Right Hand
RTD	Regional Transportation District

Acronym	Meaning
RTDM	Real Time Data Monitoring
RTN	Return
SCI	Software Configuration Items
SDD	Software Design Description
SPI	Serial Peripheral Interface
SRAM	Static Random Access Memory
SRS	Software Requirements Specification
Std	Standard
TL	Train Line
VSRS	Vapor Stone Rail Systems
W/O	Without

1.4 Document Organization

This SDD is divided into three sections and the following specific information is provided within each one:

- Section 1, Introduction, states the purpose, identifies the scope, gives definitions, abbreviations and an overview of the SDD.
- Section 2, References, lists the factors that affect the product and its requirements.
- Section 3, Software Design, summarizes followings:
    - Decomposition Description: describes functions of software entities.
    - Dependency Description: gives a brief description of the dependency between each entity.
    - Interface Description: describes the interface of the entity.
    - Detailed Design: details the functions implemented by each software entity, presents and explains tasks performed by each software entity in functional conditions.

Within the document, diagrams, flowcharts and tables are used to present the software design:

- Decision flowcharts give a graphical view of how entities and functions process work.
- The diagram Figure 3-1 describes the dependency between different modules.
- Tables contain data structures used by the Neuron SPI Module. They also present C code.

2 REFERENCES

This section lists the number, title, revision, date, and source of all documents referenced in this SDD.

Standards

IEEE Std 1016-1998	IEEE Recommended Practice for Software Design Descriptions, September 23rd 1998, IEEE, www.ieee.org
IEEE Std 1558-2004	IEEE Recommended Practice for Software Documentation for Rail Equipment and Systems, March 23rd 2005, IEEE, www.ieee.org

Vapor Stone Rail Systems Documents

CMM-01	Software Engineering Process, Rev. 3, 2001-03-31
CMM-02	Software Engineering Policy Manual, Rev. 1, 2008-07-18
S17-07	Software Design Description Template, Rev. 0, 2002-01-15[1]

[1] Complemented with IEEE STD 1558 Requirements

3 SOFTWARE DESIGN

3.1 Decomposition Description

3.1.1 Module Decomposition

3.1.1.1 Neuron SPI Module Description

The Neuron SPI Module is a single entity module performing following function:

- Send and receive messages from the Neuron Processor

3.1.2 Concurrent Process Decomposition

3.1.2.1 SPI Task Description

The two Neuron SPI task loops run every 50ms and call the internal functions. These tasks are running as an independent threads.

3.1.3 Data Decomposition

3.1.3.1 Neuron SPI Data Description

Table 3-1 and Table 3-2 show the format of SPI frames.
The maximum size for each Neuron SPI Frame is 32 bytes. If the SPI packet is larger than 32 bytes, it will be sent in multiple blocks.

Table 3-1: SPI_Frame Message from Neuron to DCM

	Bit 7							Bit 0
SYNC	1	0	1	0	1	0	1	0

	MDS_Time Bit 7							Bit 0
Data 0	MDS_Time.year (High byte)							
Data 1	MDS_Time.year (Low byte)							
Data 2	MDS_Time.month							
Data 3	MDS_Time.day							
Data 4	MDS_Time.hour							
Data 5	MDS_Time.minute							
Data 6	MDS_Time.second							
Data 7	Custom Data 1							
Data 8	Custom Data 2							
…	…							
Data n	Custom Data m							
Data n+1	ExtraDataID_FromNA							
Data n+2	ExtraData_FromNA							
Checksum	CHK7	CHK6	CHK5	CHK4	CHK3	CHK2	CHK1	CHK0

Table 3-2: SPI_Frame Message from DCM to Neuron

	Bit 7							Bit 0
SYNC	0	1	0	1	0	1	0	1
Data0	Custom Data 1							
Data1	Custom Data 2							
Data 2	Custom Data 3							
…	…							
Data k	Custom Data s							
Data k+1	ExtraDataID_ToNA							
Data k+2	ExtraData_ToNA							
Checksum	CHK7	CHK6	CHK5	CHK4	CHK3	CHK2	CHK1	CHK0

3.2 Dependency Description

3.2.1 Inter-Module Dependencies

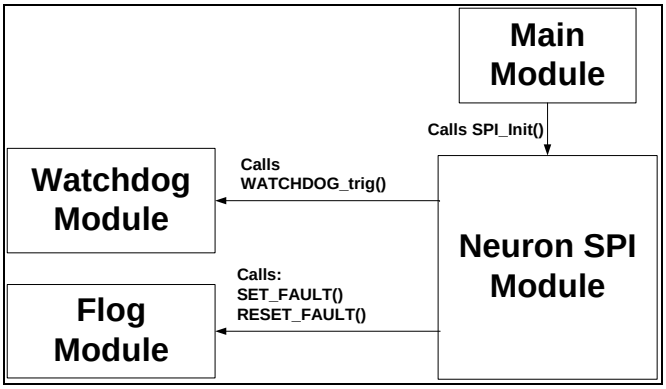

Figure 3-1: Inter-Module Dependencies Diagram

3.2.1.1 Neuron SPI Module

Figure 3-1 shows the Inter-Module Dependencies Diagram of the Neuron SPI Module. It displays Neuron SPI Module's dependency over other modules.

3.2.2 Inter-Process Dependencies

Figure 3-2 displays the two OS semaphores that Neuron SPI Module depends on:

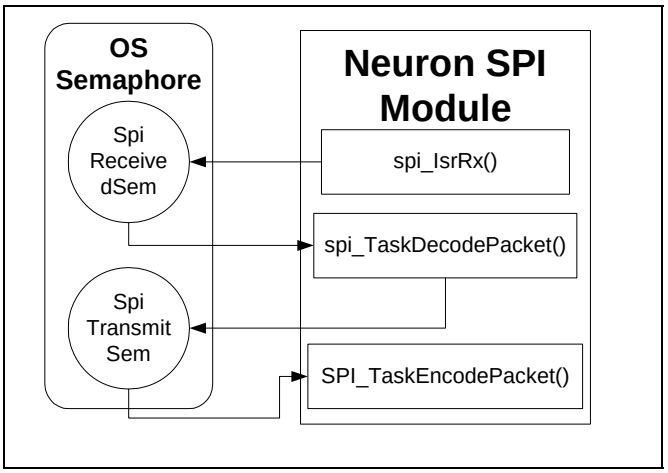

Figure 3-2: Inter-Process Dependencies Diagram

3.2.3 Data Dependencies

Not applicable.

3.3 Interface Description

This section describes the functions of Neuron SPI Module, listed in alphabetical order.

3.3.1 Module Interfaces

3.3.1.1 Neuron SPI Module External Interfaces

Function Name	void SPI_Init(int iFcpu, unsigned long ulBaud, unsigned int uiSetting);
Description	This function initializes the Neuron SPI module. It sets all necessary configurations of the SFR, depending on the selected operating mode. The configuration determines whether the SPI interrupts are to be released, and the priority of the released interrupt.
Input	iFcpu: Frequency of the CPU board in MHz ulBaud: Baud rate of the SPI Port uiSetting: Port setting
Output	None
Return	None

3.3.1.2 Neuron SPI Module Internal Interface

Function Name	void spi_IsrRx(void) interrupt SCRINT
Description	This function is an interrupt routine that is triggered at the end of packet transmission to disable SPI transmission until the next packet is ready to be transmitted. It uses the SSC for the receive interrupt service routine. (SSCRIR is set and PECC channel #2 is empty). This function sends the semaphore “SpiReceivedSem” to function spi_TaskDecodePacket at the end. (whenever a packet is received)
Input	None
Output	None
Return	None

Function Name	void spi_IsrTx(void) interrupt SCTINT
Description	This function manages the transmission interrupt service routine for the SSC. It is triggered when the sending of data is terminated (SSCTIR is set).
Input	None
Output	None
Return	None

Function Name	void spi_TaskDecodePacket (void)
Description	This task function decodes and validates a received packet from the SPI port. It waits for the semaphore “SpiReceivedSem”, and it sends the semaphore “SpiTransmitSem” to function spi_TaskEncodePacket.
Input	None
Output	None
Return	None

Function Name	void spi_TaskEncodePacket (void)
Description	Encodes a message to the SPI This function packages the transmission message. It calculates the checksum of the message. It waits for the semaphore “SpiTransmitSem” from the function spi_TaskDecodePacket.
Input	None
Output	None
Return	None

3.4 Detailed Design

This section details all operations performed by Neuron SPI Module.

3.4.1 Neuron SPI Module Detailed Design

The Neuron SPI Module has two different routines to send and receive data from the SPI port. Each of them is interrupt driven. The transmission and reception is made automatically with a Peripheral Event Controller (PEC) transfer. Each PEC channel has a source and destination pointer. The source or the destination can be incremented after each transfer. When a PEC channel is active, the PEC takes place of the standard interrupt routine for the number of transfer specified in the COUNT field of the PECCx registers. While the COUNT field has not reached zero, the transfer from the source to the destination occur in replacement of the interrupt routine. When the COUNT of a PEC channel reaches zero, the standard interrupts are then executed on the next event that triggers the interrupt.
The SPI reception is achieved with the PEC channel #2 and the transmission is achieved with the PEC channel #3. Table 3-3 shows the initial configuration for the PECC2 and PECC3. After initialization, the reception of data on the SPI port is enabled and the PEC channel 2 starts to transfer characters into the SPI_RX_IBUF in the internal RAM. A description of the SPI port and layers is shown in Figure 3-3. A maximum of 48 bytes can be received without any interruption triggered. When the COUNT value of the PECC2 register reaches zero, the functionality of the PEC is disabled and the standard interrupt routine associated with the synchronous serial port (interrupt #0xB8, SSCRINT) is executed.
The task SPI_port is executed after a complete message is received (every 50 ms) or after 100 ms to ensure that the SPI_RX_IBUF is synchronized.

Table 3-3: PEC channels Initialization for SPI Port

NAME	VALUE	DESCRIPTION	ATTRIBUTS
PECC2	0X030A	Move bytes from the receive buffer (sscrb) of spi port to internal ram	Transfer byte, increment destination, maximum of 10 bytes being transferred (0x0a)
SRCP2	SSCRB (0XF0B2)	Set the source pointer of the pecc2 to the receive buffer (sscrb) of the synchronous serial port.	
DSTP2	SPI_RX_IBUF	Set the destination pointer of the pecc2 to the designated location in internal ram.	
PECC3	0X050A	Move bytes from the internal ram to the transmit buffer (ssctb) of synchronous serial port	Transfer byte, increment source. Maximum of 10 bytes being transferred (0x0a)
SRCP3	SPI_TX_IBUF	Set the source pointer of the pecc3 to the designated location in internal ram.	
DSTP3	SSCTB (0XF0B0)	Set the destination pointer of the pecc3 to the transmit buffer (ssctb) of the synchronous serial port.	

Triggering the SSCTIR flag in the software enables the transmission. This interrupt request flag triggers the beginning of the transmission sequence using the PECC sequence.

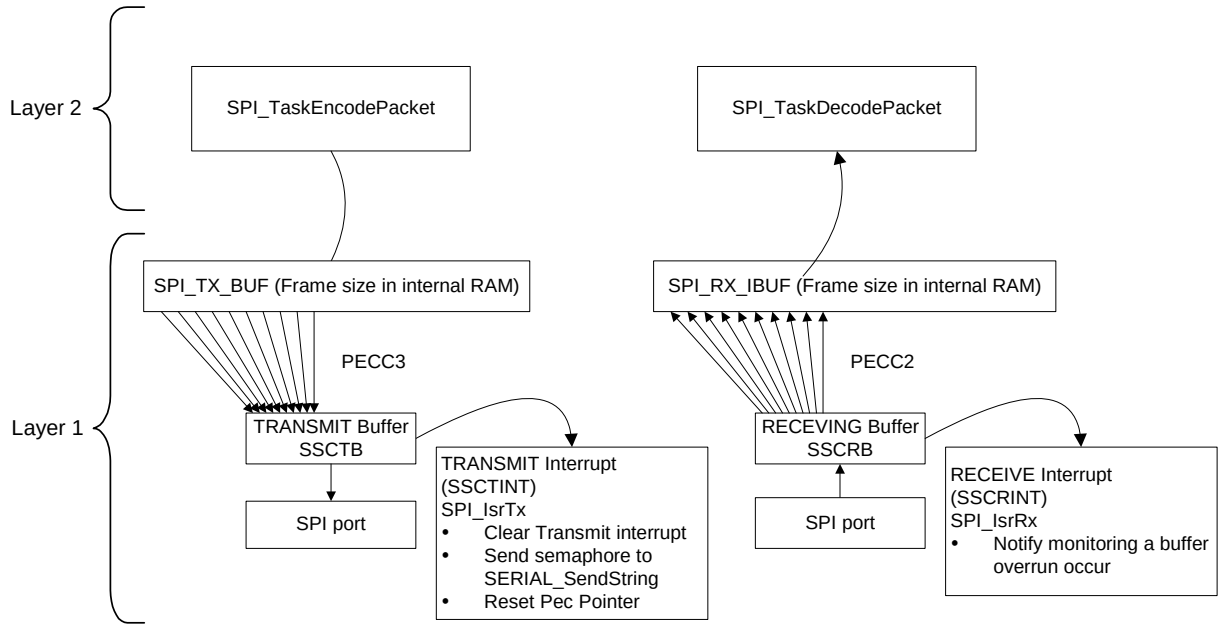

Figure 3-3: SPI Port Block Diagram

3.4.1.1 Neuron SPI Interface Description

The Neuron SPI Interface contains functions to send and receive synchronization messages from and to the SPI Port. Message received is verified and command bits are executed.
Messages that need to be sent are transferred to the SPI Port using the SPI_SendMessage function. Figure 3-4 shows the flowchart of the Neuron SPI Interface Process. If there is 10 consecutive errors or if no message is received within 2 seconds, the Event, SPI Error Detected, is set. It is reset as soon as a valid message is received.

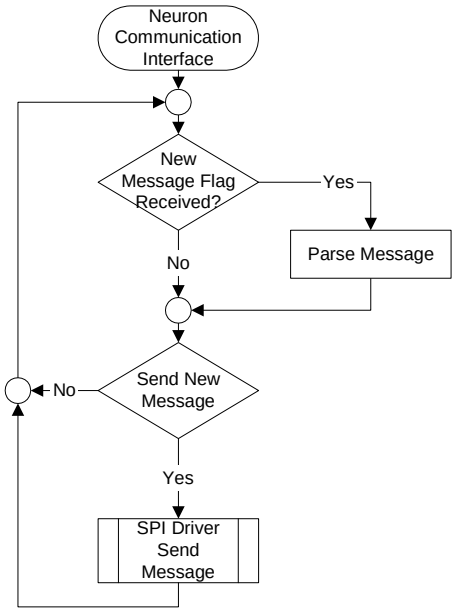

Figure 3-4: Neuron SPI Interface Process

3.4.1.2 SPI_Init Function

This function initializes CPU's SPI configuration and 2 semaphores: SpiReceivedSem and SpiTransmitSem. Then it creates two tasks functions: spi_TaskDecodePacket and SPI_TaskEncodePacket.

3.4.1.3 spi_IsrRx Function

This function is used after the last byte of the communication packet has been received. The interrupt routine sends a semaphore to the SPI_TaskPacketDecode to notify there is a new packet waiting to be decoded. The PECC channel is not reloaded in the interrupt routine and will be reloaded into SPI_TaskPacketDecode in order to decode and verify the packet.
The value of the counter is included in PECC2 register and when it goes from 1 to 0, the interrupt is then triggered and the SPI_IsrRx is executed. The interrupt flag is reset on exit.
Figure 3-5 shows the sequence:

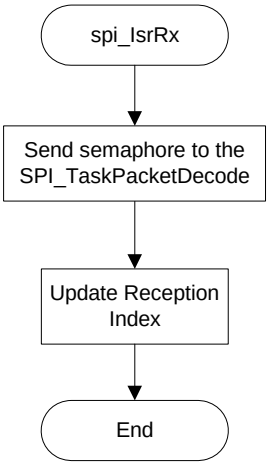

Figure 3-5: spi_IsrRx Flowchart

3.4.1.4 spi_IsrTx Function

The interrupt routine is triggered when the last byte on the SPI port is transmitted. The transmit interrupt is then disabled. The PECC channel is not reloaded in the interrupt routine and will be reloaded into SPI_TaskPacketEncode.
Figure 3-6 shows the sequence:

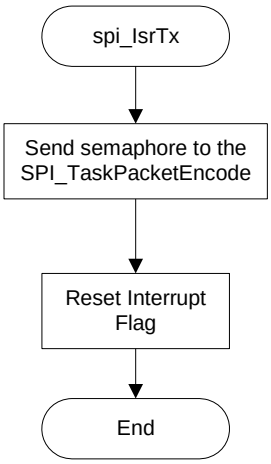

Figure 3-6: spi_IsrTx Flowchart

3.4.1.5 spi_TaskDecodePacket Function

This task function is responsible for the reception. It verifies if a packet is completed. It verifies if the first character received is the sync byte. If the first byte is not a valid header (0x82), the decoder deletes the packet and reloads the PECC to be ready to receive the next communication packet.
The SPI_TaskDecodePacket is executed every 100ms or when the task receives the semaphore (every 50 ms). If the complete message is not received within the timeout delay of 100 ms, the bytes received are then discarded and the pointers are reset to allow reception of a new message.
When the message is received within the timeout delay, a validation of the checksum is performed. If the checksum doesn't match the one calculated, the message is discarded and the pointers are reset to allow reception of new messages. Each invalid communication increments an error counter in order for the monitoring module to detect a broken communication link between main CPU and Neuron CPU.
In case the checksum calculated matches the one received, the task extracts the data from the packet received, and sends the packet to the IO module.
At the end, it triggers the watchdog and suspends itself until another packet is received.

Figure 3-7 shows the sequence:

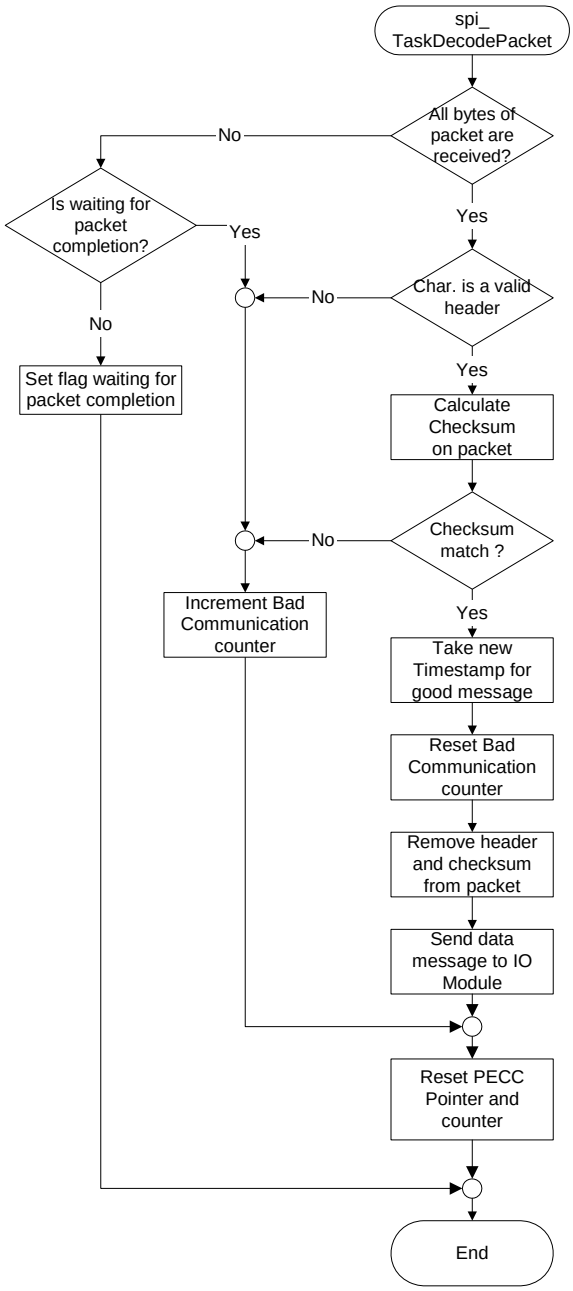

Figure 3-7: spi_TaskDecodePacket Flowchart

3.4.1.6 spi_TaskEncodePacket Function

This task function prepares data to be transmitted to the Neuron Processor. The function receives the message to be transmitted that contains only the data. It fills the SPI_TX_BUF, adds the header, and then calculates the checksum. The PECC in SPI_TX_BUF is adjusted and the preparation of the PEC #3 starts the transmission. Figure 3-8 shows the sequence:

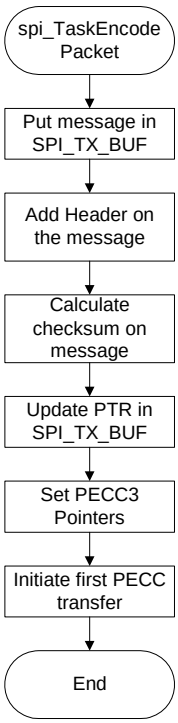

Figure 3-8: spi_TaskEncodePacket Flowchart

3.4.2 Data Entities Detailed Design

Not applicable.